

03. Linear Filters

(Cross-correlation, Convolution)

Jae-Young Choi

Sungkyunkwan University

Outline

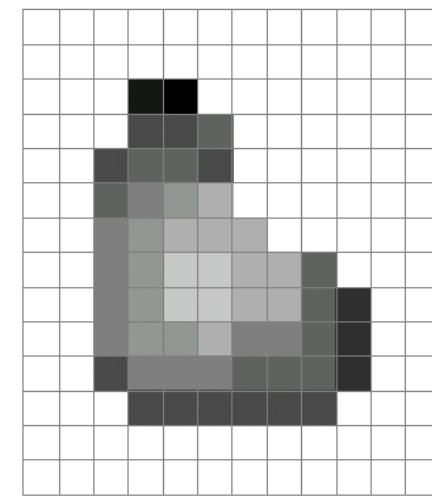
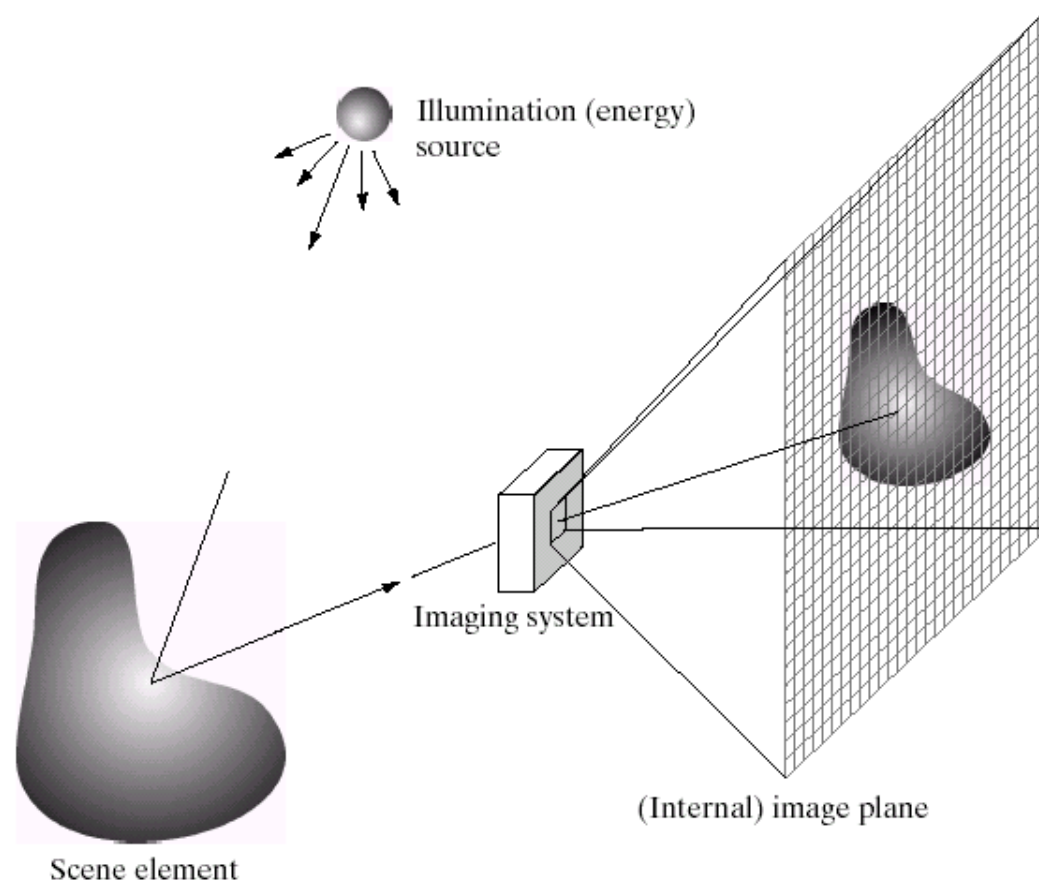


- What is an image
- Linear filter
- Convolution
- Related Codes

What is an image?



What is an image?

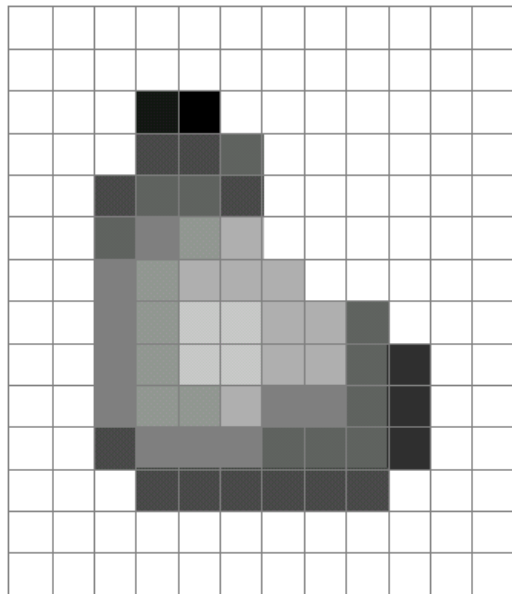


Digital image

Source: A. Efros

What is an image?

- A grid (matrix) of intensity values



=

255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	20	0	255	255	255	255	255	255	255
255	255	255	75	75	75	255	255	255	255	255	255
255	255	75	95	95	75	255	255	255	255	255	255
255	255	96	127	145	175	255	255	255	255	255	255
255	255	127	145	175	175	175	255	255	255	255	255
255	255	127	145	200	200	175	175	95	255	255	255
255	255	127	145	200	200	175	175	95	47	255	255
255	255	127	145	145	175	127	127	95	47	255	255
255	255	74	127	127	127	95	95	95	47	255	255
255	255	255	74	74	74	74	74	74	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255

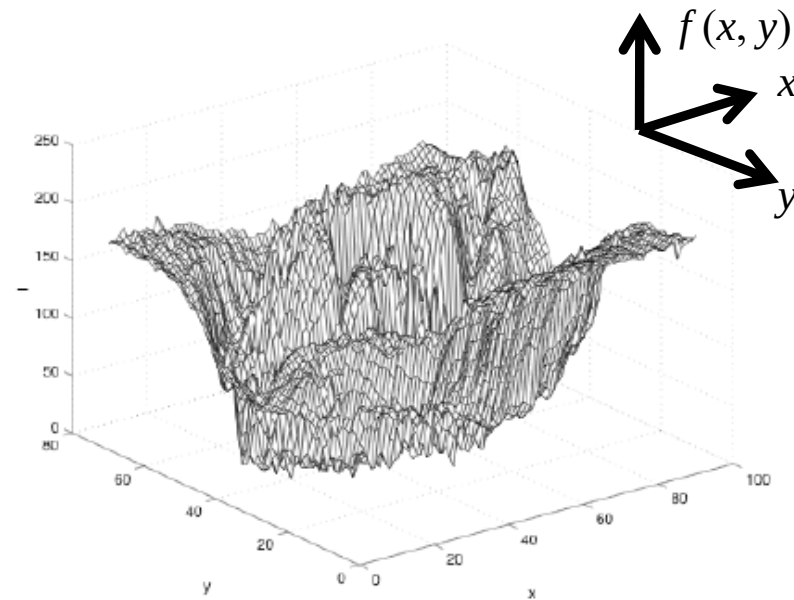
(common to use one byte per value: 0 = black, 255 = white)

What is an image?

- We can think of a (grayscale) image as a **function**, f , from $\mathbb{R}^2$ to $\mathbb{R}$:
 - $f(x,y)$ gives the **intensity** at position (x,y)



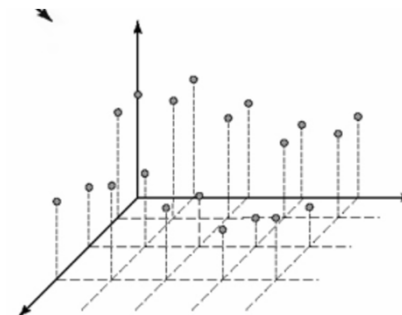
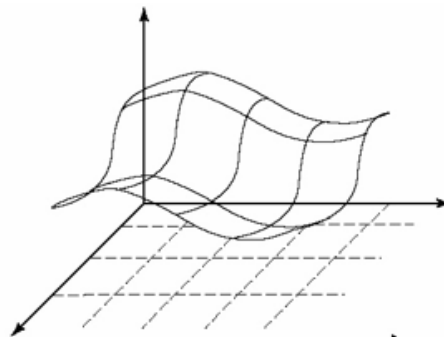
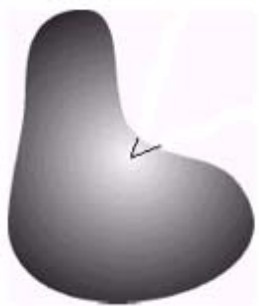
Image patch



The range of the image function f

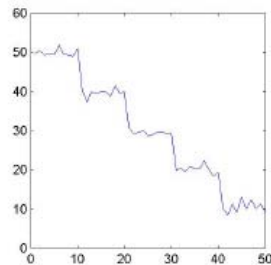
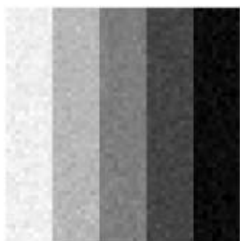
What is an image?

- Digital image: a two-dimensional light intensity function $f(x,y)$ where x and y denote spatial coordinates, the value of f at any point is proportional to the brightness (gray level) of the image at that point.
- A digital image:
 - is discretized in the spatial domain
 - Is discretized in the brightness domain.



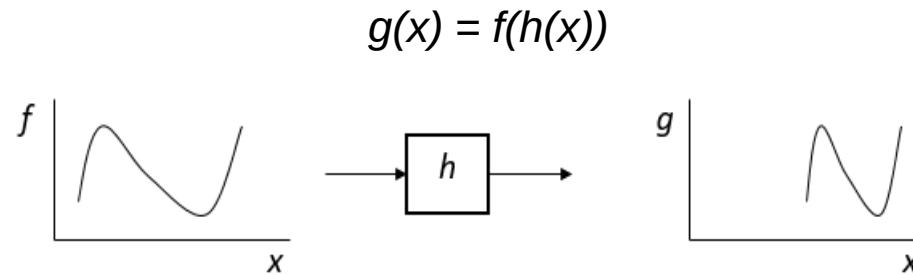
What is an image?

- Images contain noise – sources sensor quality, light fluctuations, quantization effects
- Given a noisy image
 - How do we reduce noise ?
 - How do we find useful features ?

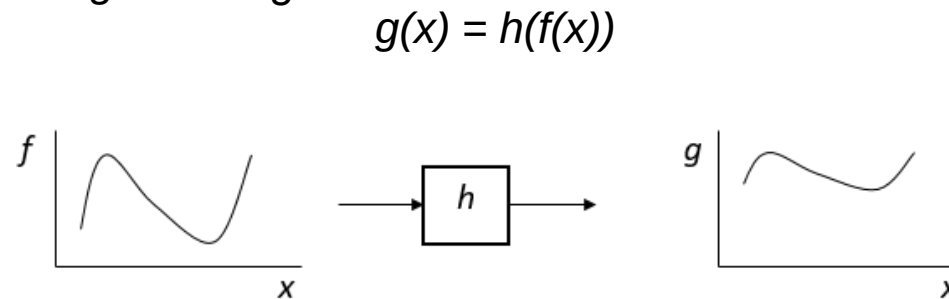


What is an image?

- image warping: change *domain* of image



- image filtering: change *range* of image



What is an image?

- Filtering

- Form a new image whose pixels are a combination of the original pixels

- Why?

- Extract useful information from the images
 - Features (edges, corners, blobs, ...)
- Modify or enhance image properties
 - de-noising, Super-resolution, in-painting

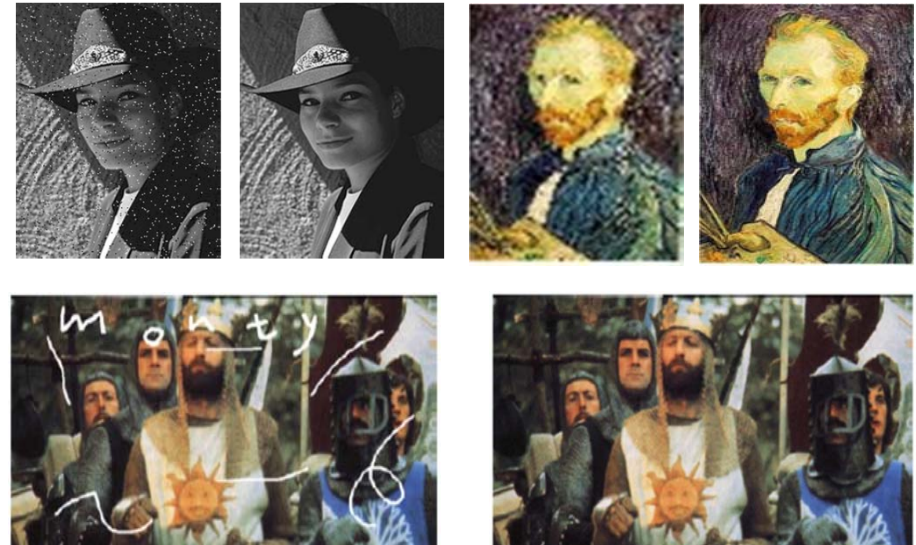
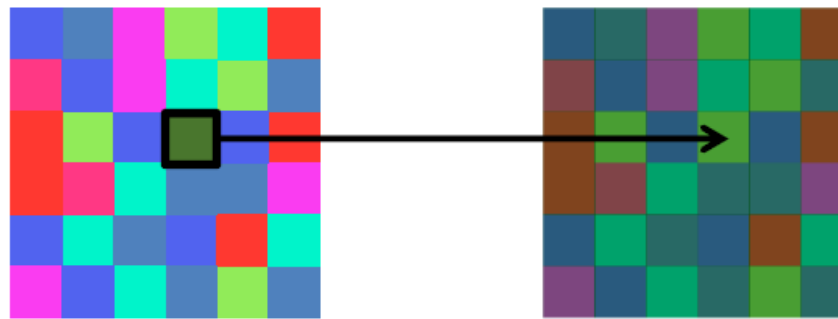


Image Filtering (Cross-correlation)

Types of Image Filtering

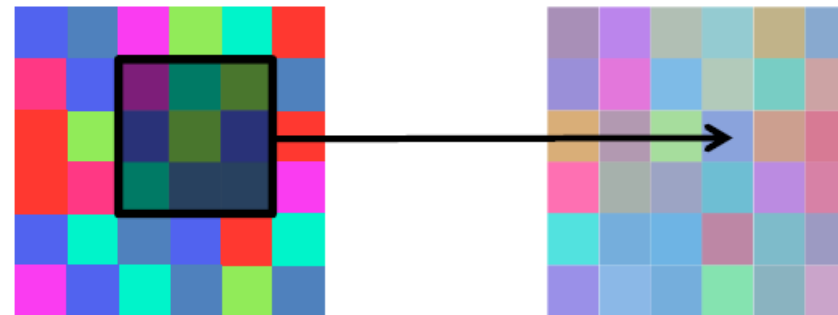
- What operations can we perform on pixels?

- Point processing



Point Operation

- Filters



Neighborhood Operation

Types of Image Filtering

■ Point Operations

- Point Operations perform a mapping of the pixel values without changing the size, geometry, or local structure of the image
- Each new pixel value $I'(u,v)$ depends on the previous value $I(u,v)$ at the same position and on a mapping function $f(\cdot)$
- The function $f(\cdot)$ is independent of the coordinates
- Such operation is called “homogeneous point operations”

$$\begin{aligned}a' &\leftarrow f(a) \\ I'(u, v) &\leftarrow f(I(u, v))\end{aligned}$$

Image Filtering

■ Filters

- Point operations are limited (why)
- They cannot accomplish tasks like sharpening or smoothing
- We need a function that involves the intensities (color) in the neighborhood of each pixel

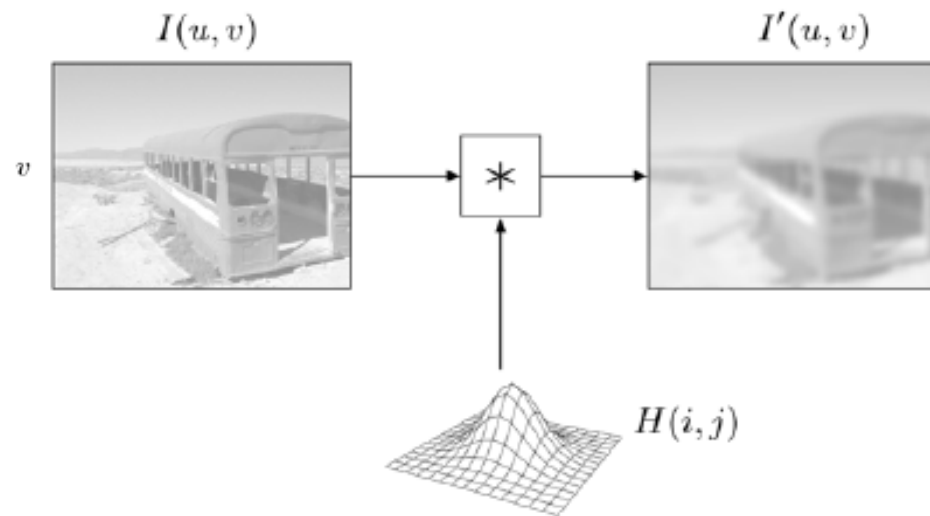


Image Filtering

- Linear Shift-Invariant Image Filtering
 - Replace each pixel by a *linear* combination of its neighbors
 - The combination is determined by the filter's *kernel*.
 - The same kernel is *shifted* to all pixel locations so that all pixels use the same linear combination of their neighbors

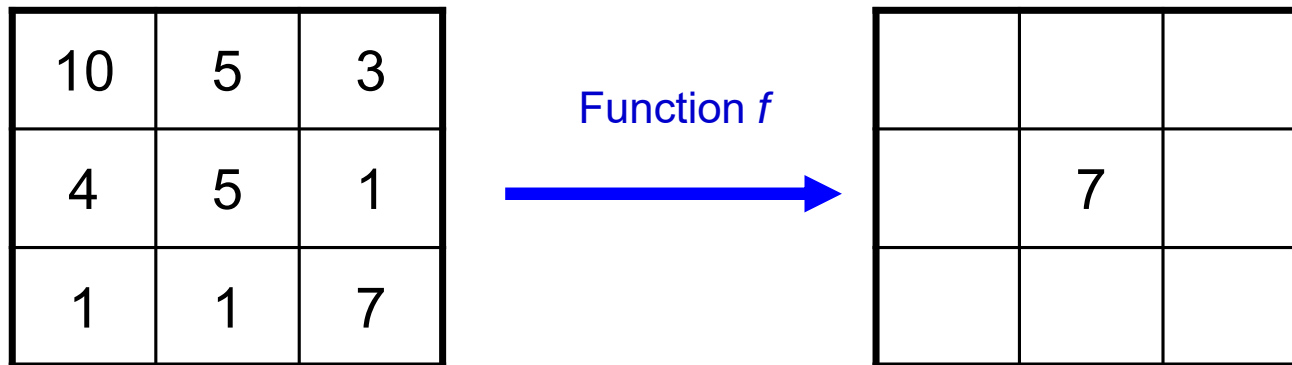


Image Filtering

- Linear case is the simplest and most useful
 - Form a new image by replacing each pixel with a weighted sum (i.e., linear combination) of its neighbors, using the same set of weights at each point

10	5	3
4	5	1
1	1	7

 $*$

0	0	0
0	0.5	0
0	1.0	0.5

 $=$

	7	

kernel

Image Filtering

- Ex) 2D discrete-space moving average with 3×3 window
- Let f be the image, h be the kernel (of size $2k+1 \times 2k+1$), and g be the output image

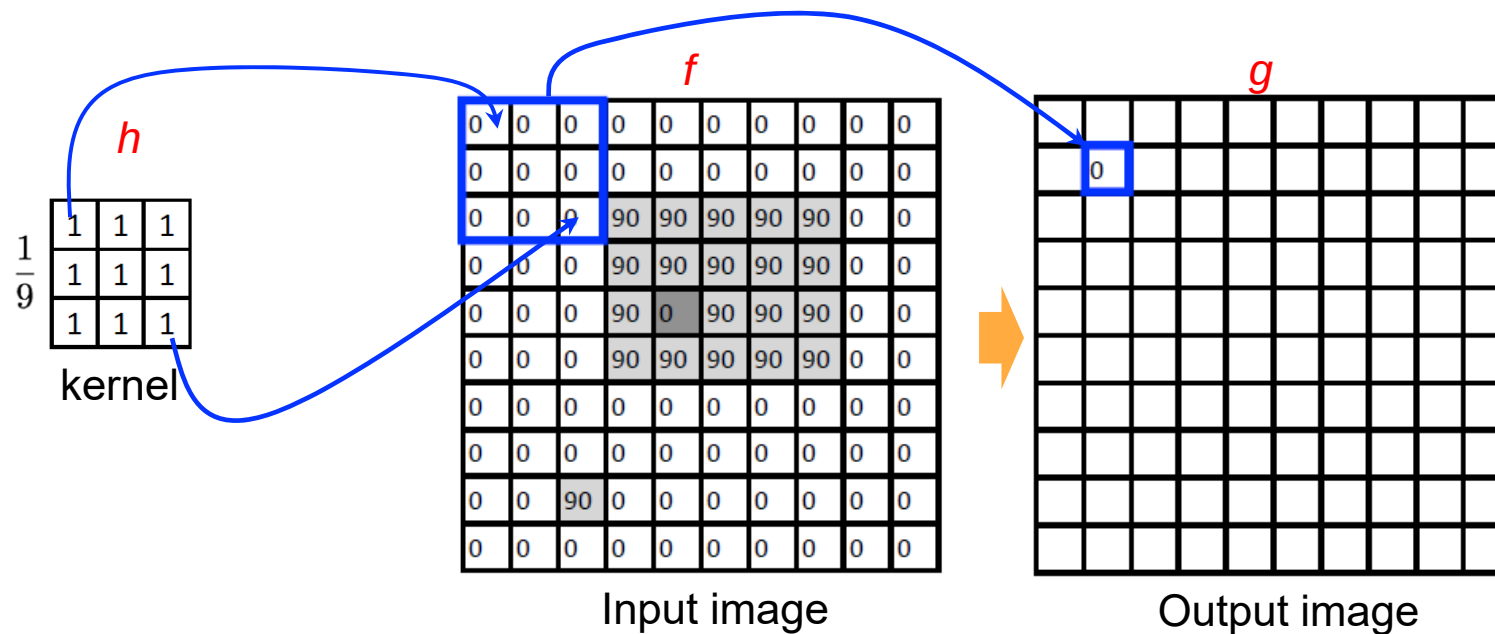
$$g[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] f[i + u, j + v]$$

$$g = h \otimes f$$

Image Filtering

- Square Mean filter

- replaces pixel with local average. it has smoothing (blurring) effect



$$g[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] f[i + u, j + v]$$

Image Filtering

- Square Mean filter

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1

kernel

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Input image



	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	0	10	20	30	30	30	20	10	
	10	10	10	10	0	0	0	0	
	10	10	10	10	0	0	0	0	

Output image

Image Filtering

- Square Mean filter

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Input image



	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	0	10	20	30	30	30	20	10	
	10	10	10	10	0	0	0	0	
	10	10	10	10	0	0	0	0	

Output image



kernel

1	1	1
1	1	1
1	1	1

$\frac{1}{9}$



Image Filtering

- What is the size of the output?
 - Full : output size is sum of sizes of f and h
 - Same : output size is same as f
 - Valid : output size is difference of sizes of f and h

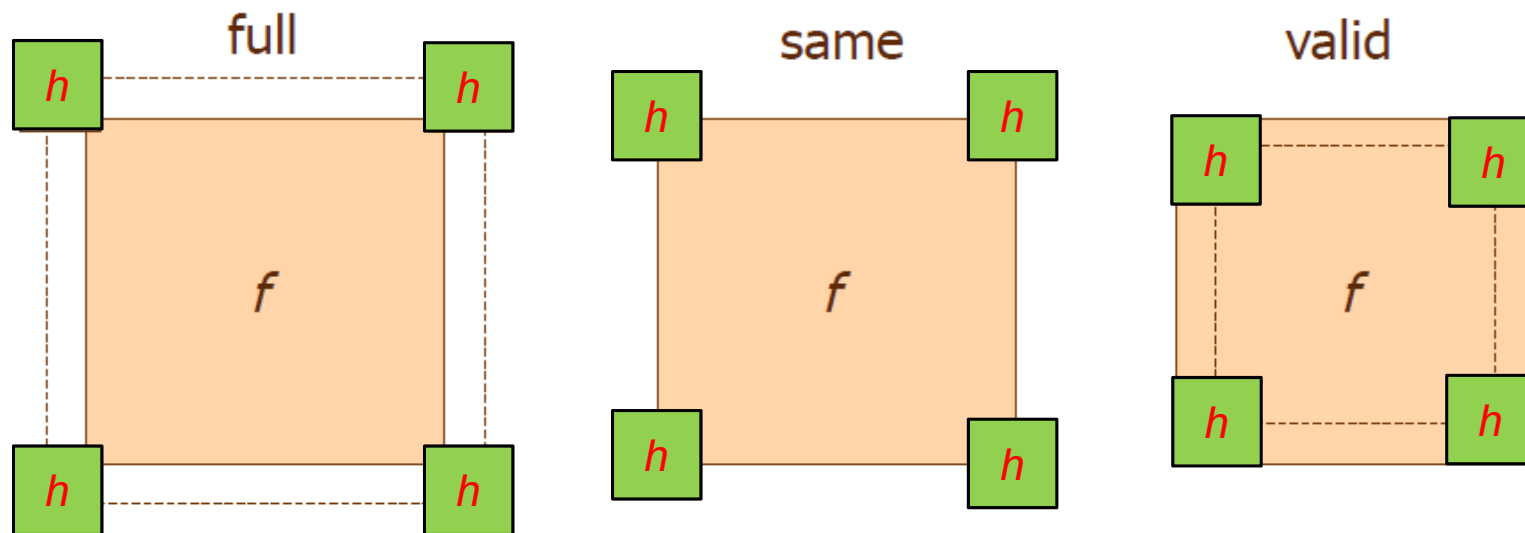


Image Filtering

- What is the size of the output?
 - Same : output size is same as f $\Rightarrow$ “pad” is needed

$$p_s = (h_s - 1) / 2$$

- Valid : output size is difference of sizes of f and h

$$g_s = f_s - h_s + 1$$

Image Filtering

- Linear filter's key properties
 - **Linearity** : $\text{filter}(h_1 + h_2) = \text{filter}(h_1) + \text{filter}(h_2)$
 - **Shift invariance**: same behavior regardless of pixel location:
 $\text{filter}(\text{shift}(h)) = \text{shift}(\text{filter}(h))$
 - Theoretical result: any linear shift-invariant operator can be represented as a convolution

Linear filter examples



Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Linear filter examples



Original

0	0	0
0	0	1
0	0	0



Shifted left
by 1 pixel with
correlation

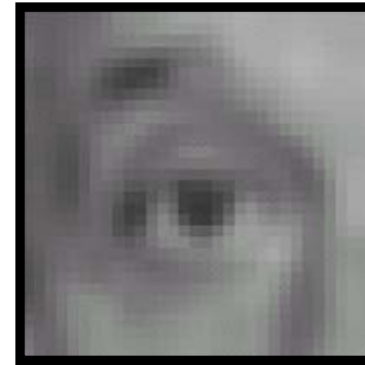
Linear filter examples



Original

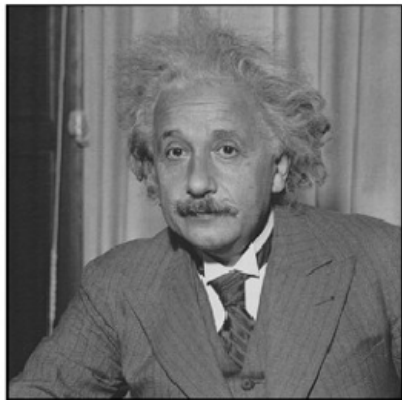
$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1



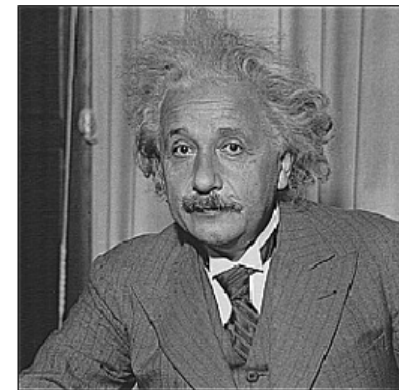
Blur (with a box filter)

Linear filter examples



Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Sharpening



Image Filtering

- How big should the mask be?
- The bigger the mask,
 - more neighbors contribute.
 - smaller noise variance of the output.
 - bigger noise spread.
 - more blurring.
 - more expensive to compute.

Image Filtering

- Ex) Smoothing by Averaging
- What is wrong with the picture ?



Image Filtering

- Smoothing with box filter revisited
- What's the solution?
 - To eliminate edge effects, weight contribution of neighborhood pixels according to their closeness to the center

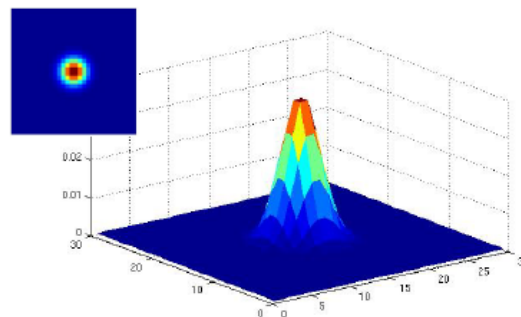
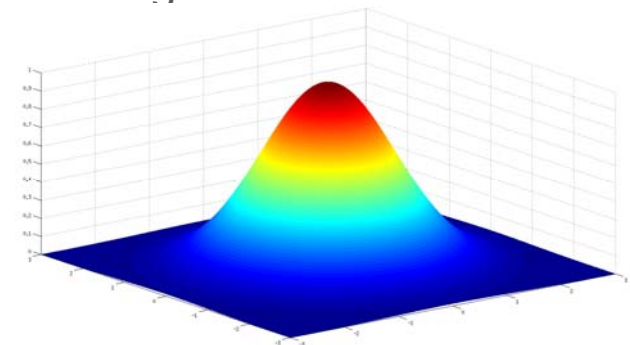


Gaussian Filter

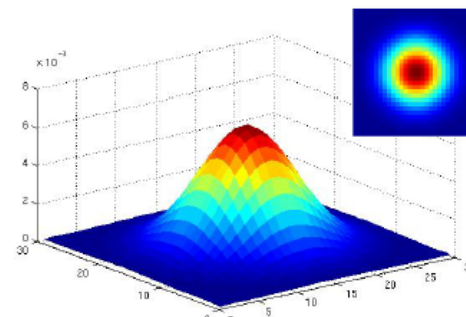
- A particular case of averaging
 - The coefficients are samples of a 2D Gaussian.
 - Gives more weight at the central pixel and less weights to the neighbors.
 - The further away the neighbors, the smaller the weight.

$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$

- Standard deviation σ : determines extent of smoothing



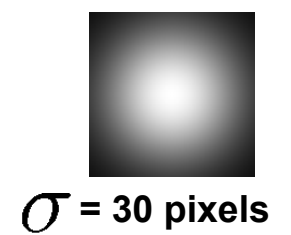
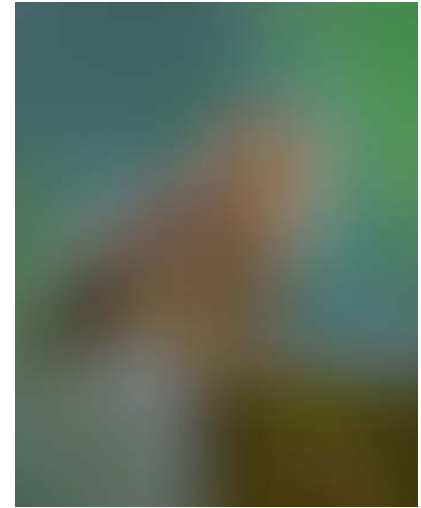
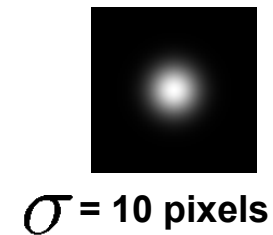
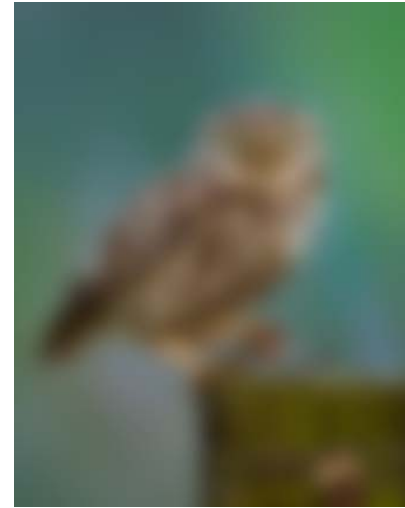
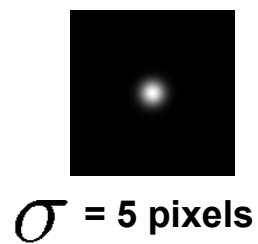
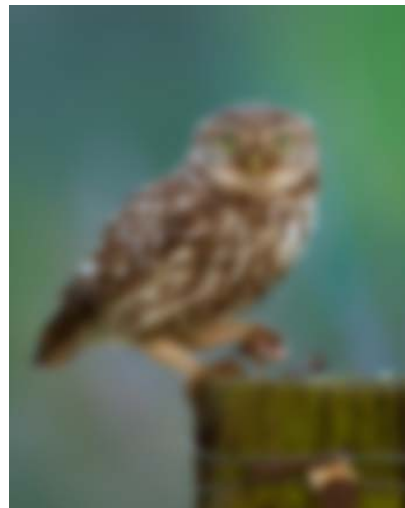
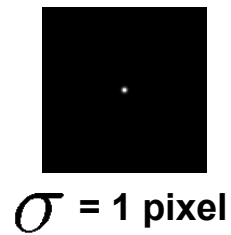
$\sigma = 2$ with 30 x 30
filter



$\sigma = 5$ with 30 x 30
filter

Gaussian Filter

- The Gaussian function has infinite support, but discrete filters use finite kernels

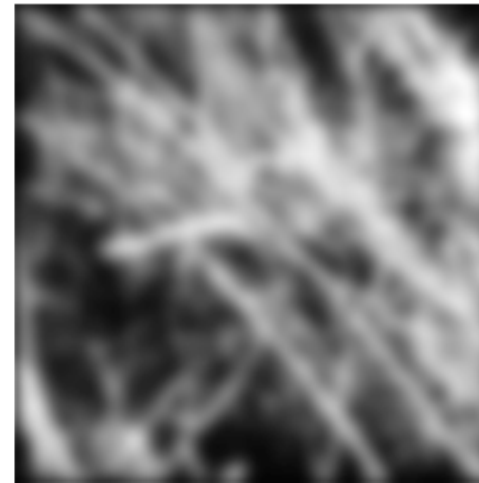
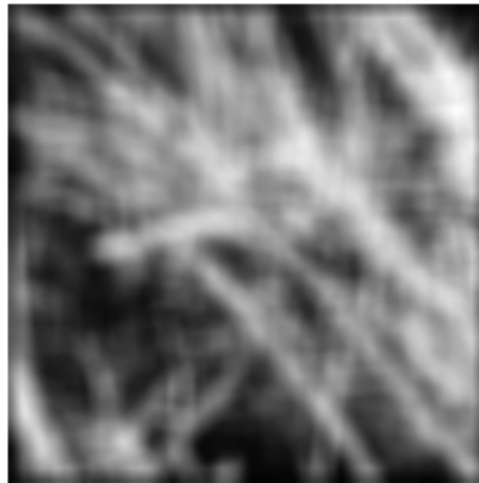


Gaussian Filter

- Advantages of Gaussian filtering
 - rotationally symmetric (for large filters)
 - filter weights decrease monotonically from central peak, giving most weight to central pixels
 - Simple and intuitive relationship between size of σ and the smoothing.
 - Convolution of a Gaussian with itself is another Gaussian.

Gaussian Filter

- Smoothing with a Gaussian



Separability

- Advantage of separability

- First convolve the image with a one dimensional horizontal filter
- Then convolve the result of the first convolution with a one dimensional vertical filter
- For a $k \times k$ Gaussian filter, 2D convolution requires k^2 operations per pixel
- But using the separable filters, we reduce this to $2k$ operations per pixel.

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

Separability

■ Complexity of filtering

- What is the complexity of filtering an $n \times n$ image with an $m \times m$ kernel?

$O(n^2 m^2)$

- What if the kernel is separable?

$O(n^2 m)$

1	2	1
---	---	---

2	3	3
3	5	5
4	4	6

11		
18		
18		

1
2
1

11
18
18

65		

1	x	1 2 1	=	1 2 1	2 3 3	= 2 + 6 + 3 = 11
2				2 4 2	3 5 5	= 6 + 20 + 10 = 36
1				1 2 1	4 4 6	= 4 + 8 + 6 = 18
						65

Non-linear Filtering

- **Salt and pepper noise:**
 - contains random occurrences of black and white pixels
- • **Impulse noise:** contains
 - random occurrences of white pixels
- • **Gaussian noise:** variations in
 - intensity drawn from a Gaussian normal distribution



Original



Salt and pepper noise



Impulse noise



Gaussian noise

Non-linear Filtering

- Reducing salt-and-pepper noise
- What's wrong with the results?

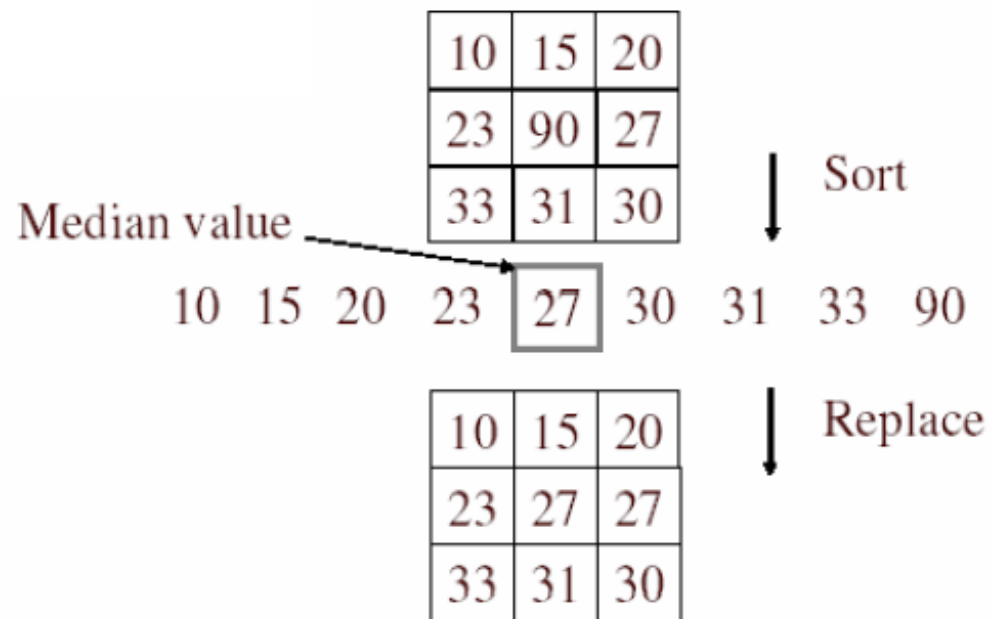


Non-linear Filtering

- Replace each pixel with the MEDIAN value of all the pixels in the neighborhood
- Non-linear
- Does not spread the noise
- Can remove spike noise
- Expensive to run

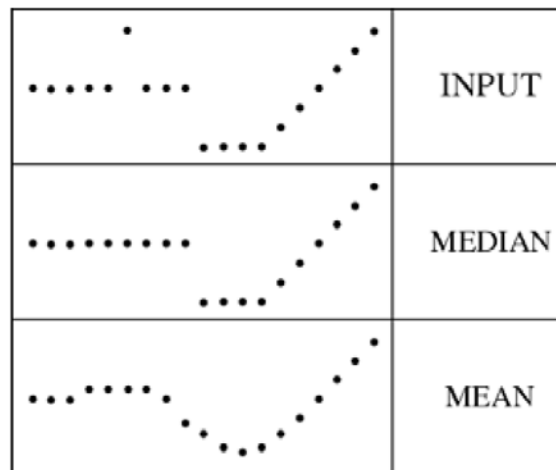
Non-linear Filtering

- Replace each pixel with the MEDIAN value of all the pixels in the neighborhood

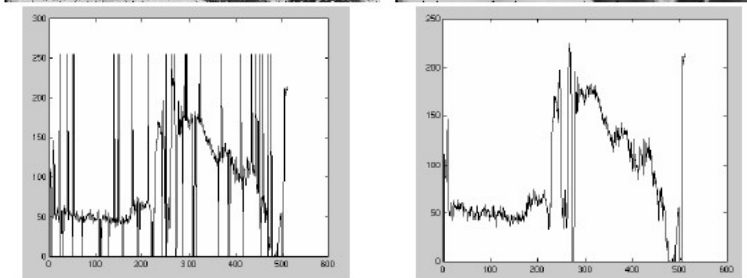


Non-linear Filtering

- Advantage of Median filter
 - Remove spikes
 - Good for impulse, salt & pepper noise
 - Less sensitive to outliers compared to mean filter



Salt and pepper noise Median filtered



Plots of a row of the image

Non-linear Filtering

- Gaussian vs. median filtering

Gaussian

3x3



5x5



7x7



Median



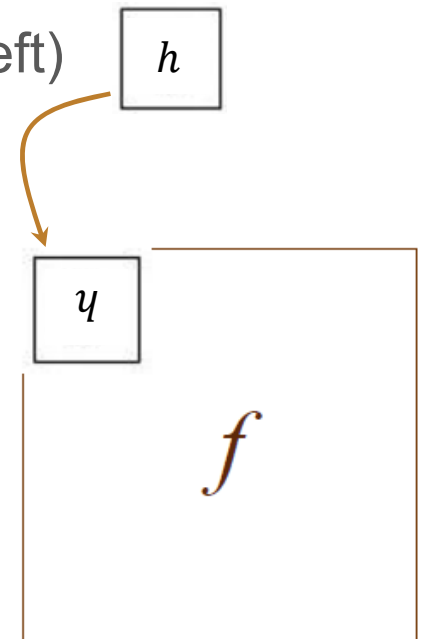
Image Filtering (Convolution)

Convolution Filtering

- Let f be the image, h be the kernel (of size $2k+1 \times 2k+1$), and g be the output image
- Flip the filter in both dimensions (bottom to top, right to left)
- Then apply cross-correlation

$$g[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] f[i - u, j - v]$$

$$g = h \star f$$



Convolution Filtering

- Convolution:

$$g[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] f[i - u, j - v]$$

$$g = h \star f$$

- Cross-correlation:

$$g[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] f[i + u, j + v]$$

$$g = h \otimes f$$

Convolution Filtering

- Convolution:

- Integral that expresses the amount of overlap of one function as it is shifted over another function
- Convolution is a filtering operation

- Cross-correlation:

- Computes a measure of similarity of two input signals as they are shifted by one another
- The correlation result reaches a maximum at the time when the two signals match best
- Correlation is a measure of relatedness of two signals

Convolution Filtering

■ Properties of Convolution

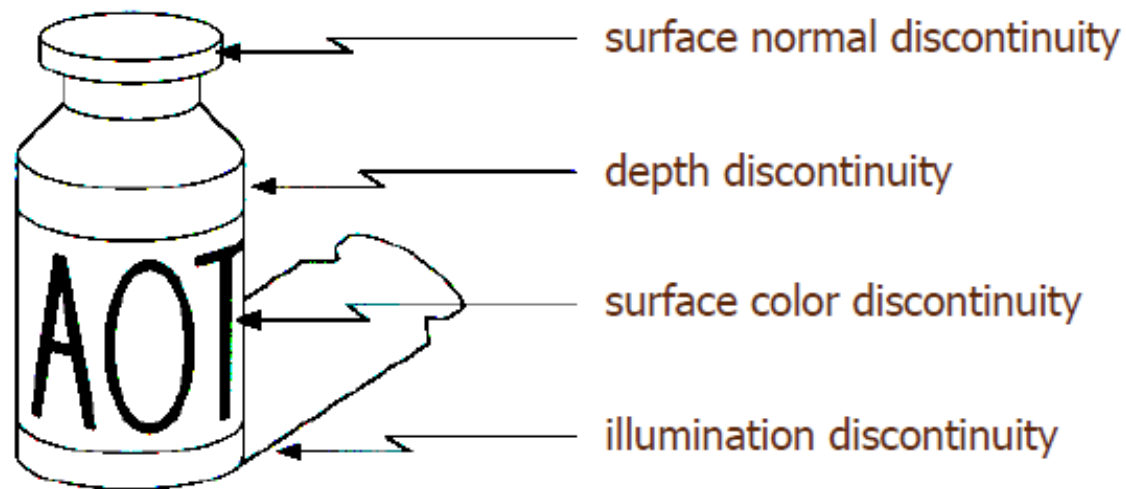
- Commutativity : $f * g = g * f$
- Associativity : $f * (g * h) = (f * g) * h$
- Distributivity : $f * (g + h) = (f * g) + (f * h)$
- Associativity with scalar multiplication :

$$a(f * g) = (af) * g \quad (\text{for any real or complex number } a)$$

- Differentiation : $\frac{d}{dx}(f * g) = \frac{df}{dx} * g = f * \frac{dg}{dx}$

Edge detecting

- Edges are caused by a variety of factors :

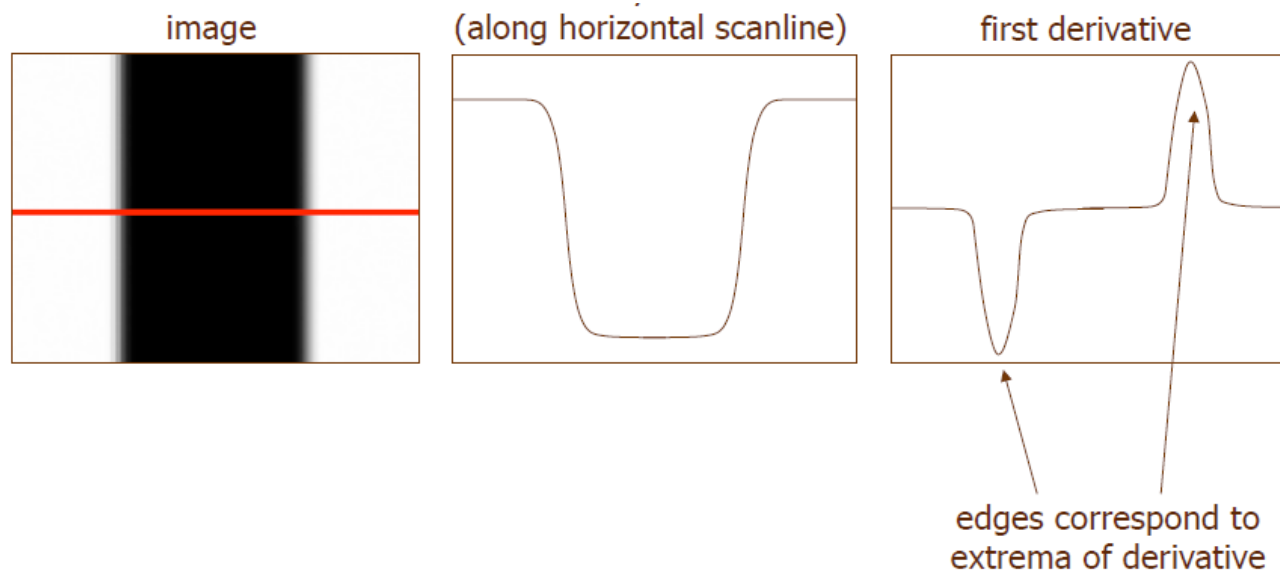


Source: Steve Seitz

Edge detecting

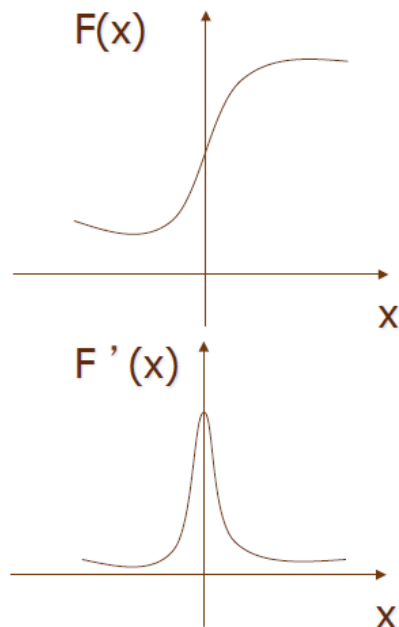
- Characterizing edges

- An edge is a place of rapid change in the image intensity function



Edge detecting

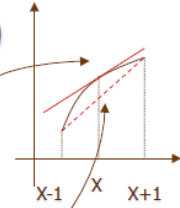
- Edge detection (1D)



Edge= sharp variation



Large first derivative

$$\frac{df(x)}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

$$\frac{df(x)}{dx} \cong \frac{f(x+1) - f(x-1)}{2}$$



Convolve with:

-1	0	1
----	---	---

Edge detecting

- Edge detection (2D)

Horizontal Edges:

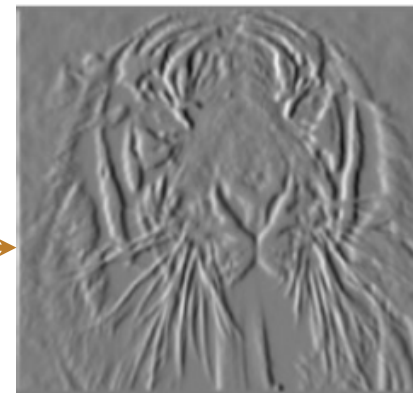
Convolve with:

-1
0
1

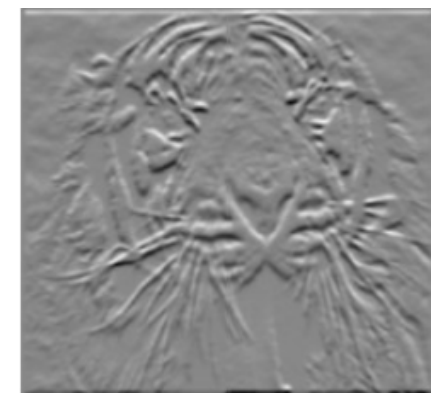
Vertical Edges:

Convolve with:

-1	0	1
----	---	---



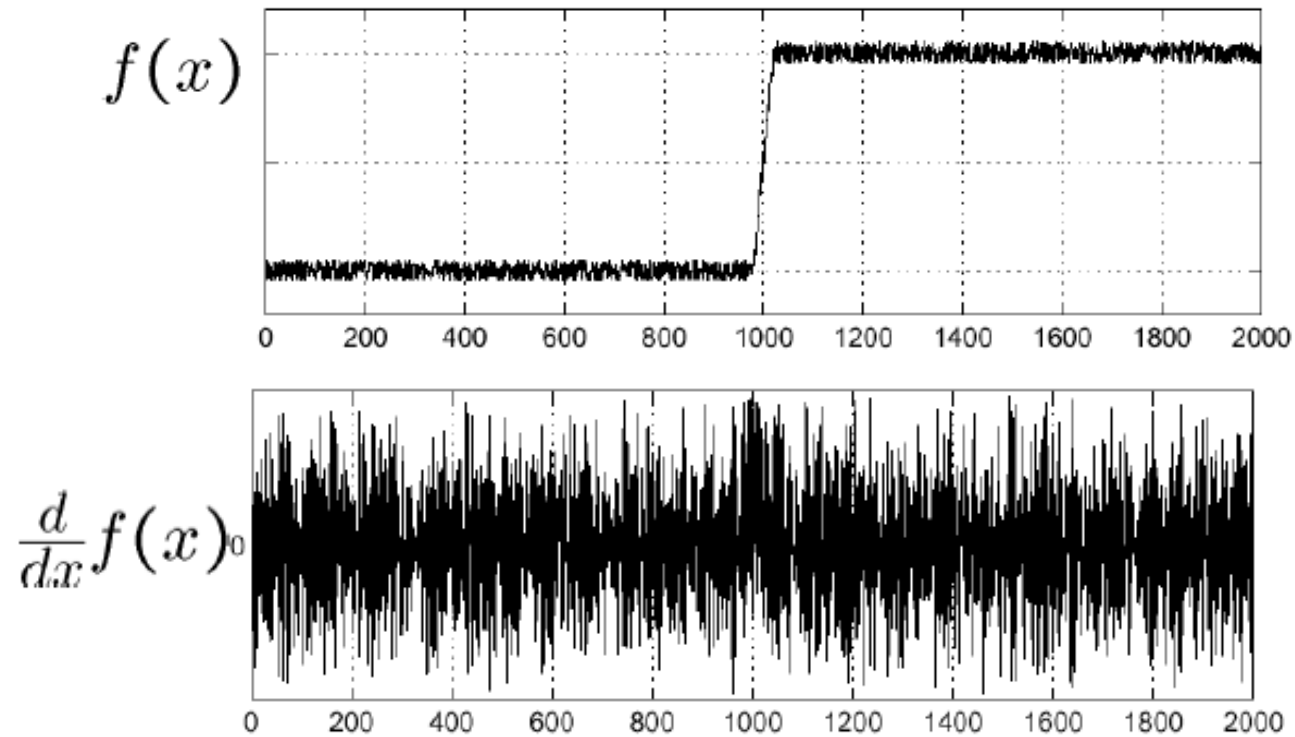
$$\frac{\partial f(x, y)}{\partial x}$$



$$\frac{\partial f(x, y)}{\partial y}$$

Edge detecting

- Effects of noise

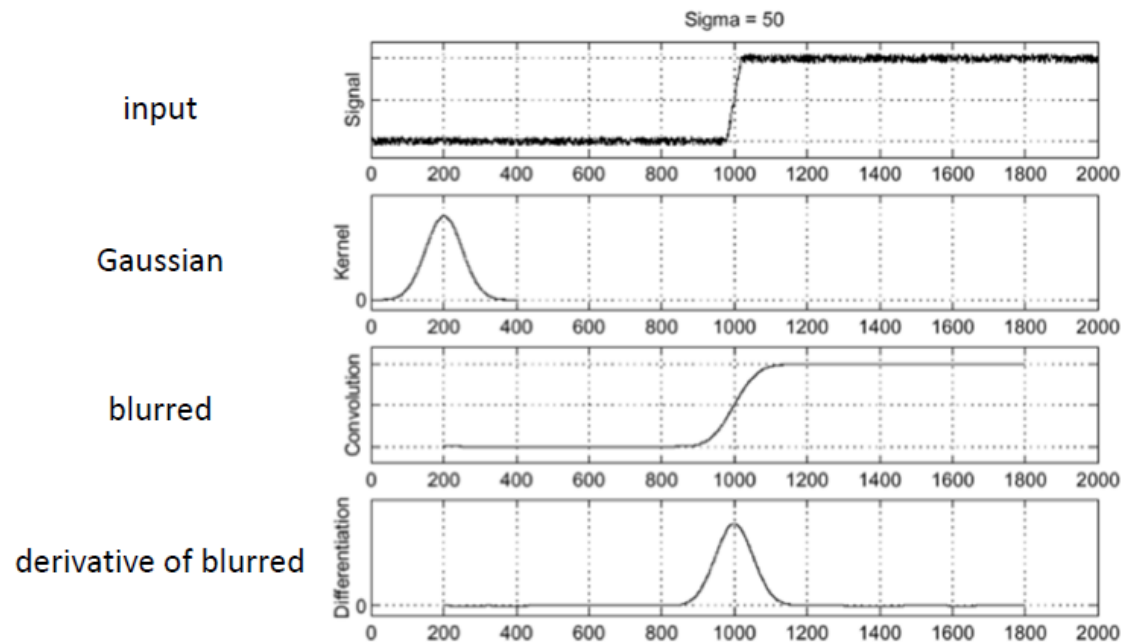
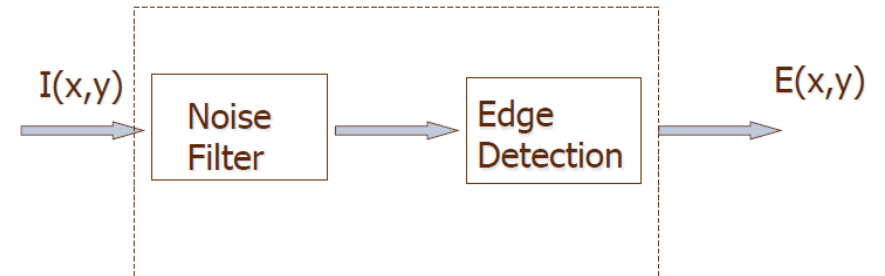


Where is the edge?

Edge detecting

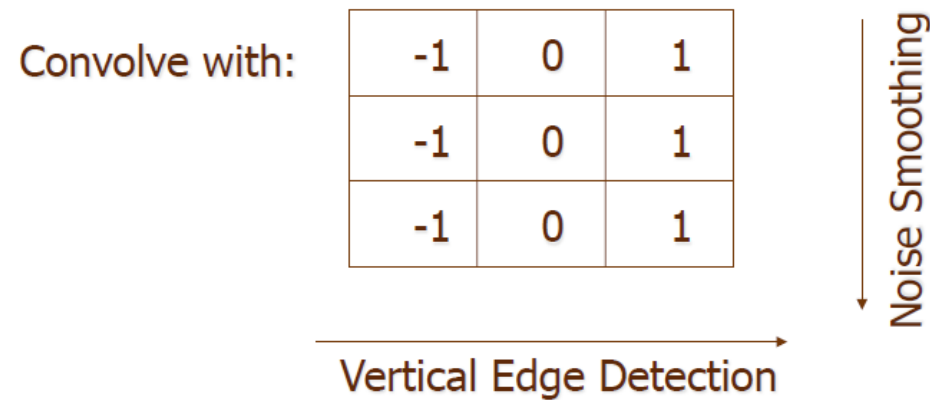
■ Noise cleaning and Edge Detection

- We need to also deal with noise
- Combine Linear Filters



Edge detecting

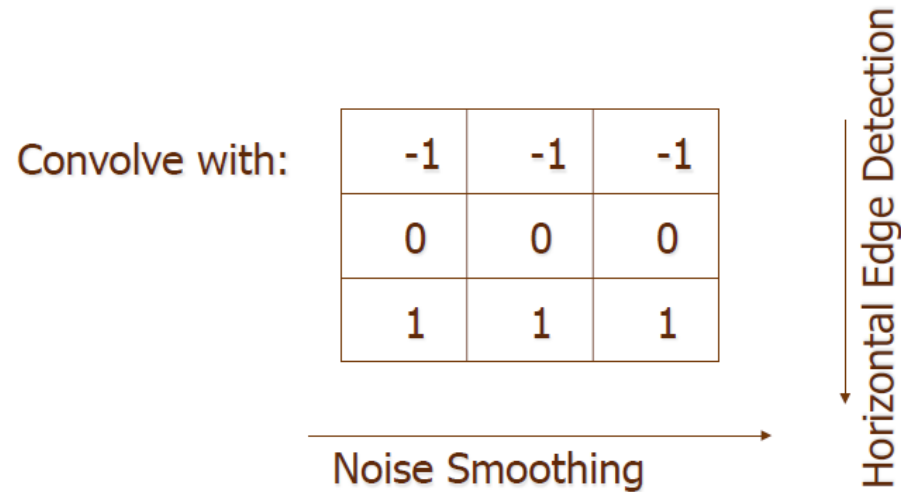
- Noise cleaning and Edge Detection



- This mask is called the (vertical) **Prewitt Edge Detector**
- Outer product of box filter $[1 \ 1 \ 1]^T$ and $[-1 \ 0 \ 1]$

Edge detecting

- Noise cleaning and Edge Detection



- This mask is called the (horizontal) **Prewitt Edge Detector**

Edge detecting

- Sobel Edge Detector

- Gives more weight to the 4-neighbors

Convolve with:

-1	0	1
-2	0	2
-1	0	1

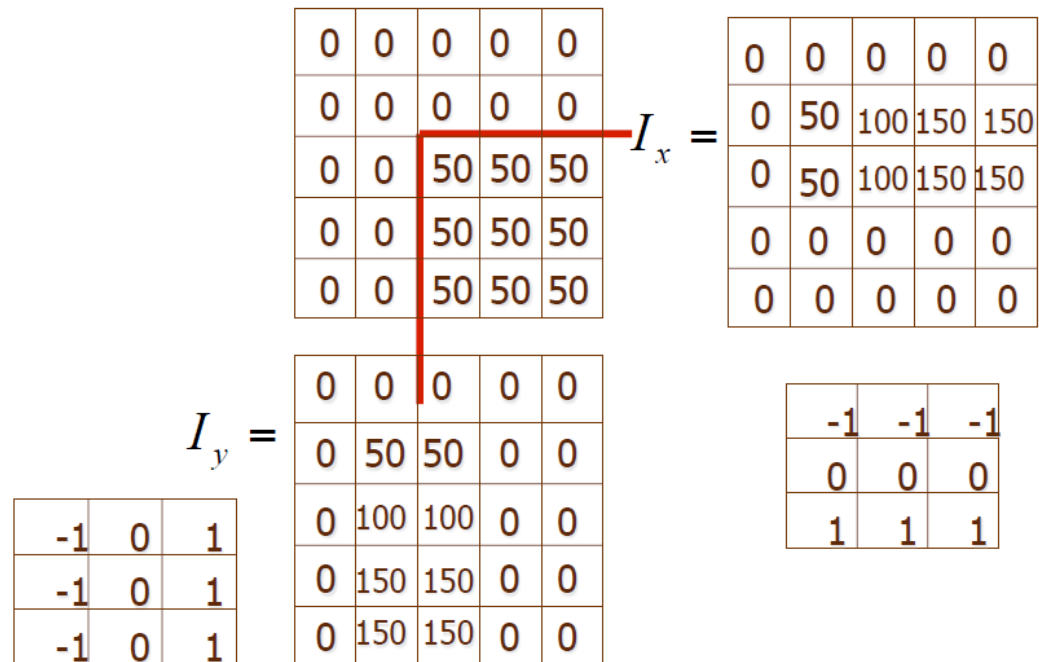
and

-1	-2	-1
0	0	0
1	2	1

Gives more weight
to the 4-neighbors

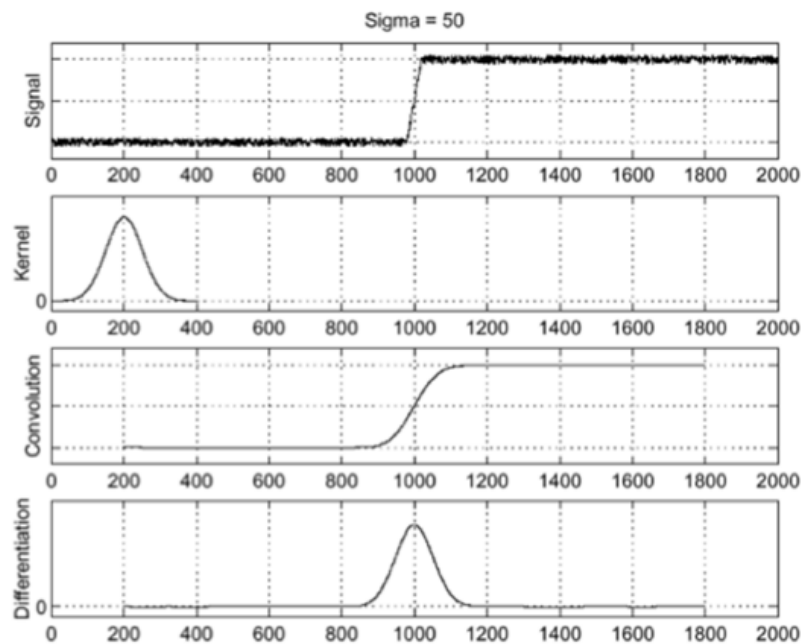
Edge detecting

- Sobel Edge Detector
 - example

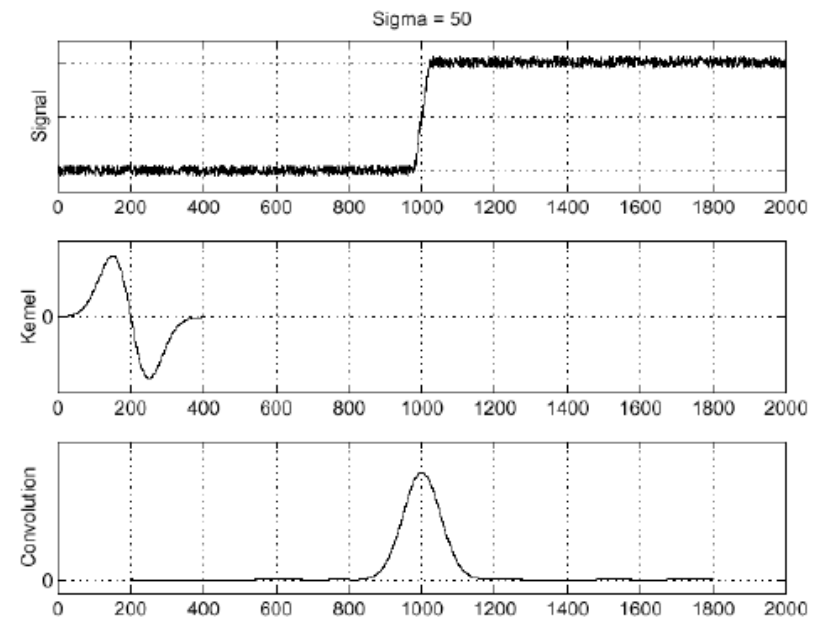


Derivative of Gaussian filter

- Derivative theorem of convolution
 - Differentiation is convolution, and convolution is associative
 - This saves us one operation



vs.

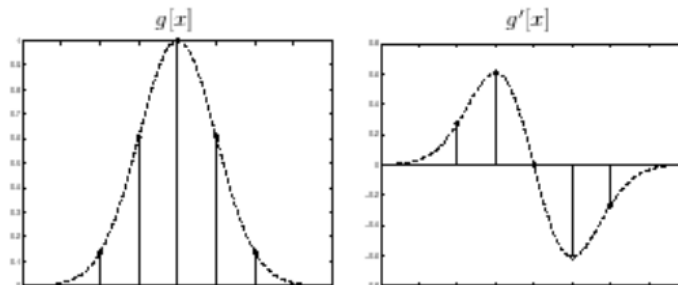


Derivative of Gaussian filter

- Taking a derivative – linear operation (take the derivative of the filter)

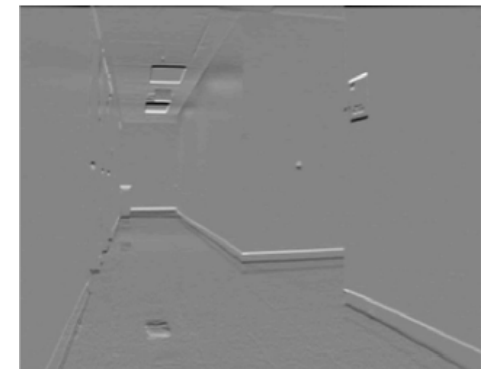
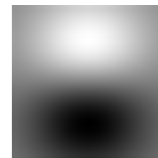
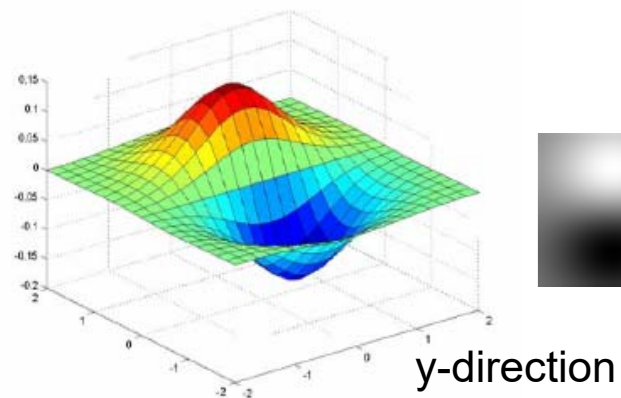
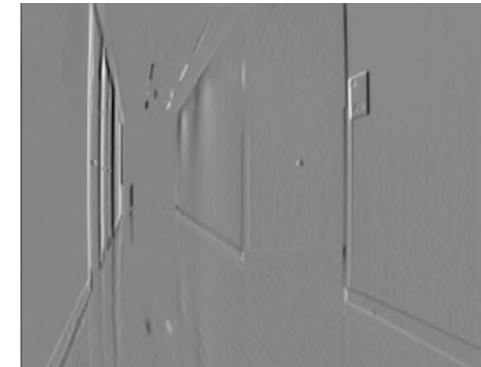
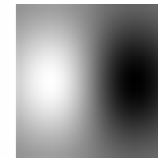
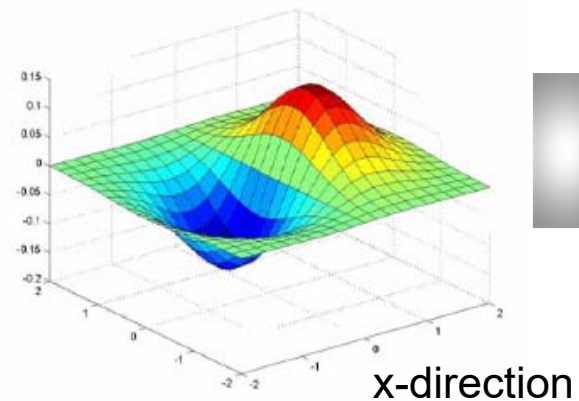
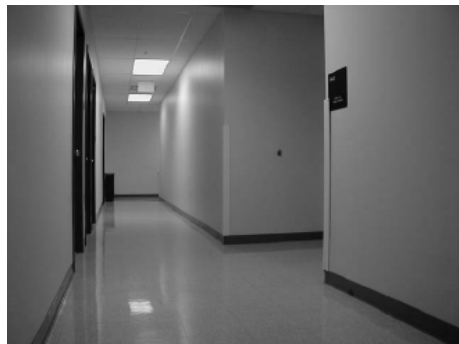
$$I_x[x, y] = I[x, y] * g'[x] * g[y] = \sum_{k=-\frac{w}{2}}^{\frac{w}{2}} \sum_{l=-\frac{w}{2}}^{\frac{w}{2}} I[k, l] g'[x - k] g[y - l],$$

$$I_y[x, y] = I[x, y] * g[x] * g'[y] = \sum_{k=-\frac{w}{2}}^{\frac{w}{2}} \sum_{l=-\frac{w}{2}}^{\frac{w}{2}} I[k, l] g[x - k] g'[y - l].$$



$$g(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{\frac{-x^2}{2\sigma^2}}, \quad g'(x) = -\frac{x}{\sigma^2\sqrt{2\pi}\sigma} e^{\frac{-x^2}{2\sigma^2}}.$$

Derivative of Gaussian filter



Derivative of Gaussian filter

- DoG



original



Derivative of Gaussian

Derivative of Gaussian filter

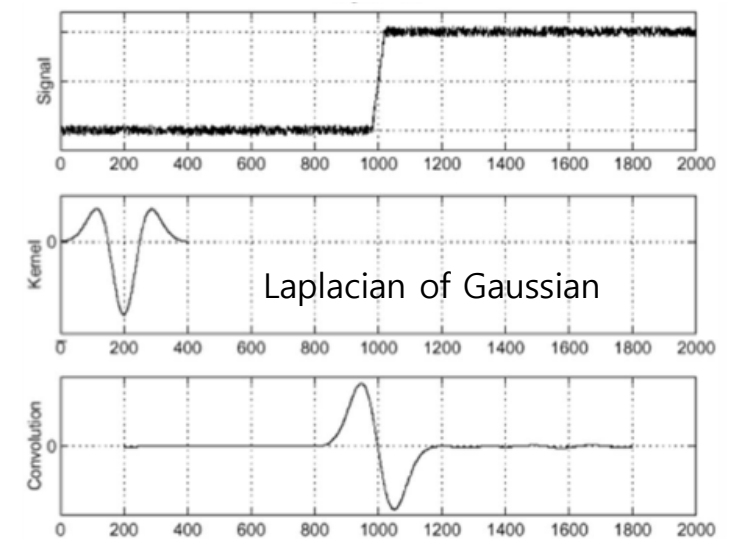
- DoG vs LoG(Laplacian of Gaussian)



Derivative of Gaussian



Laplacian of Gaussian



Related Codes

■ Week 2 – Image Transformation

- Geometric Transformations of Images :

https://docs.opencv.org/master/da/d6e/tutorial_py_geometric_transformations.html

■ Week 3 – Image Filtering

- Image Gradients (Sobel, Laplacian) :

https://docs.opencv.org/master/d5/d0f/tutorial_py_gradients.html

- Sobel, Laplacian edge detection : <https://youtu.be/b-K-vA1eiMA>

- <https://youtu.be/CJMCoAsK-h0> (using moving pictures)

Thanks!